Journal of Real-Time Image Processing (2025) 22:159
https://doi.org/10.1007/s11554-025-01738-3

RESEARCH

Efficient real-time license plate recognition using deep learning on edge devices

Fedi Sonnara[1] · Hamadi Chihaoui[1] · Fethi Filali[1]

Received: 14 October 2024 / Accepted: 16 July 2025 / Published online: 30 July 2025

Abstract
Real-time automatic license plate recognition (ALPR) is essential for smart-traffic, tolling, parking, and policing, yet roadside cameras must run on < 10 W hardware with limited memory and patchy connectivity, ruling out cloud off-loading. These constraints demand compact, fast models resilient to oblique views, motion blur, glare, and diverse plate styles. We introduce *Light-Edge*, a single-pass deep network that jointly localizes plates and recognizes characters. It shares a ResNet-18 + FPN backbone, removes 28 % of convolutions with a 1×1 channel-fusion block, and replaces anchors with an anchor-free head followed by a CTC decoder. After mixed-precision compilation in Torch-TensorRT, the 38 MB model sustains 14 FPS on a Jetson Nano—73 % faster than the anchor-free AF-Net (8.1 FPS) and 49 % faster than YOLOv8-MobileLPR (9.5 FPS)—while keeping competitive accuracy (90.2 % mAP) and halving AF-Net's power consumption (4.8 W vs 8.8 W). Light-Edge therefore satisfies the stringent speed–accuracy envelope required for large-scale, privacy-preserving ALPR on resource-constrained edge devices.

Keywords Automatic license plate recognition (ALPR) · Object detection · Deep learning · End-to-end model

1 Introduction

Automatic license plate recognition (ALPR) underpins a wide spectrum of smart-city services—including vehicle-access control for private parking facilities and low-emission zones, payment operations such as tolling, EV charging and ticket-less parking, security and law enforcement tasks like speed or red-light enforcement and mobile/fixed surveillance, and large-scale traffic data gathering for smart-city analytics—by automatically extracting plate characters from still images or video frames [7]. Although modern detectors perform well with controlled lighting and high-quality optics, they often falter in the wild, where complex backgrounds, glare, rain, fog, motion blur and occlusion are common. These uncontrolled conditions, together with the requirement for real-time inference on edge devices that operate below 10 W, demand models that balance accuracy with computational efficiency.

Traditional pipelines and multi-stage deep networks achieve high accuracy in laboratory settings but struggle with latency and robustness when deployed on resource-constrained cameras. Our work therefore targets a light-weight, end-to-end deep model that delivers real-time performance on edge hardware without sacrificing accuracy or generalization.

Modern ALPR roadside cameras typically embed boards such as an NVIDIA Jetson Nano (128-core Maxwell GPU, quad-core A57 CPU, 4 GB RAM, ≤ 10 W TDP) or a Raspberry Pi 4, leaving < 512 MB of usable GPU memory and a hard latency budget of ≈ 100 ms per frame. These limits preclude two-stage ALPR pipelines with duplicate backbones, anchor matching or CPU-GPU shuttling. Light-Edge is engineered around these realities: (i) one shared ResNet-18 + FPN backbone eliminates redundant feature extractors; (ii) an anchor-free head removes IoU matching and per-anchor NMS; and (iii) INT8 Torch-TensorRT compilation fuses kernels, cutting power to 4.8 W and raising throughput to 14

Open Access funding provided by Qatar National Library.

✉ Fethi Filali
filali@qmic.com

Fedi Sonnara
sonnara.fedi@gmail.com

Hamadi Chihaoui
chihaoui.hamadi@gmail.com

[1] QMIC, Qatar University, Doha, Qatar

FPS—comfortably inside the 10 FPS / 10 W envelope and tripling throughput-per-watt over FP32 baselines (Table 2).

Despite recent progress, ALPR still struggles in uncontrolled settings, as illustrated in Fig. 1. Critical issues are: *(i)* recognition accuracy; *(ii)* compactness and speed on edge hardware; *(iii)* plate tilt and motion blur; *(iv)* poor image quality and weather effects; and *(v)* plate-style diversity (color, font, size, character count).

ALPR comprises two tightly coupled tasks:

License plate detection: The goal is to localize plates via bounding boxes or corner points (Fig. 2). Methods span: *traditional* (edge / morphology), *CNN-based* detectors, and *end-to-end* networks that unify detection and recognition.

License plate character recognition: Given a detected plate, characters are decoded using the same three method families. Recognition can be *segmentation-based*, which first isolates characters (Fig. 3), or *segmentation-free*, which reads the plate holistically.

Segmentation-based pipelines are sensitive to noise and illumination; segmentation-free CNNs alleviate this but often treat detection and recognition as separate stages, increasing redundancy and latency.

End-to-end ALPR models [5, 9, 13] reduce error propagation, yet prior versions either lack generalization or miss real-time targets on edge devices. Our work therefore develops a lightweight, end-to-end network optimized for the Jetson Nano, achieving near-real-time inference without compromising accuracy.

Fig. 1 Challenging CCPD samples. The figure gathers ten thumbnails from the CCPD dataset [13], each illustrating a nuisance factor that can defeat naïve ALPR pipelines. *Top row—left → right:* (**a**) headlamp glare and windshield reflections; (**b**) extreme low-light/night driving; (**c**) distant plate amid background clutter; (**d**) heavy cast shadow; (**e**) partial occlusion by another object. *Bottom row—left → right:* (**f**) long-range view in dense traffic; (**g**) specular reflections on a glossy hood; (**h**) large in-plane rotation; (**i**) strong out-of-plane rotation; (**j**) snow and dirt contamination. Taken together, these snapshots highlight the wide appearance variations an ALPR system must cope with and motivate the design of detectors and recognizers that are robust to illumination, pose, occlusion and weather

Fig. 2 Bounding box vs. corner points. A CCPD sample [13] with the axis-aligned box (red) for fast localization and the four-corner polygon (green) for pose-aware rectification prior to recognition

The remainder of this paper is structured as follows: Sect. 2 discusses related works, highlighting the key differences between various ALPR techniques. Section 3 delves into the model design, detailing the architecture and the components employed. Section 4 focuses on the model's training and optimization processes to ensure efficient performance. Section 5 presents the results achieved by the proposed model, comparing its performance to existing approaches. Finally, Sect. 6 offers concluding remarks and outlines potential avenues for future work in the ALPR domain.

2 Related work

Research on automatic license plate recognition spans three generations of systems: (i) *traditional*, handcrafted pipelines that pre-date CNNs; (ii) *deep learning multi-stage* frameworks that run detection and recognition sequentially; and (iii) *end-to-end* networks that unify both tasks in a single model. We review each strand in turn, highlighting detection and recognition techniques, key benchmarks, and their practical trade-offs—and conclude each subsection by contrasting it with our proposed *Light-Edge* architecture, which seeks the accuracy of end-to-end CNNs while meeting the strict speed-and-power envelope of edge devices.

2.1 Traditional ALPR techniques

Before the advent of CNNs, ALPR relied on multi-stage pipelines driven by handcrafted descriptors that worked

Fig. 3 Character segmentation. Original plate (top) and its binarized mask (bottom) used to isolate individual glyphs before OCR [3]

only under controlled illumination. For plate *detection* these systems exploited the rectangular outline, distinctive colors and fixed-length text of a licence plate: edge-based EM clustering and line-density filtering with color salience and an SVM [20, 33]; color-template matching, later robustified with wavelet + EMD texture cues [17, 30]; multi-level extended LBP for texture discrimination under variable lighting [26]; and character-centric approaches that assemble glyphs detected by MSER or CRF [15, 29]. Although each family offers a specific speed–robustness compromise, all degrade sharply in the presence of glare, motion blur or occlusion—limitations that the CNN detector in our Light-Edge model overcomes by learning features directly from diverse data.

Recognition engines of the same era first segmented individual characters and then classified them via template matching, SVMs, HMMs or hybrid RBMs [16, 19]. Segmentation-free OCR avoided cropping, whereas segmentation-based variants relied on connected components or extremal regions followed by handcrafted descriptors such as SIFT. Contemporary CNN recognizers replace these stages with an end-to-end network that learns optimal features from pixels and predicts the full string without explicit segmentation, at the expense of larger datasets and compute. Light-Edge adopts this latter philosophy: an anchor-free detector feeds shared features straight to a CTC decoder, eliminating the brittle segmentation step while remaining lightweight enough for edge deployment.

2.2 Deep learning multi-stage ALPR

Convolutional neural networks (CNNs) replaced handcrafted pipelines by tackling detection and recognition in two successive stages. Detectors use either *two-stage* region—proposal networks or faster *one-stage* heads that regress boxes directly; recognizers then decode cropped plates via segmentation-based CNN classifiers, segmentation-free sequence models, or end-to-end decoders. These systems achieve strong accuracy under glare, blur and occlusion, yet their duplicated feature extractors inflate parameters, demand large annotated datasets and slow edge device deployment.

2.2.1 License plate detection

Modern detectors share a backbone—DarkNet, ResNet, VGG, DenseNet—before an anchor-based or anchor-free head predicts the box. Lightweight backbones with <20 layers reduce latency [28]. Anchor-based YOLO [6], SSD [12] and Faster R-CNN [11] balance speed and accuracy; YOLO variants underpin robust ALPR in [27] and handle rotated plates in [25]. Anchor-free schemes streamline training by removing IoU matching [24]; cascades for tiny plates [23] and RNN-augmented heads [22] further lift recall. *Light-Edge adopts an anchor-free head atop a ResNet-18 + FPN backbone, then adds a* 1×1 *channel-fusion block that trims 28 % of convolutions-yielding a 38 MB detector that runs at 14 fps on Jetson Nano.*

2.2.2 License plate recognition

Segmentation-free recognition now dominates: CNNs classify characters directly [34], combine deep features with kernel-ELM [32], or segment by semantics via FCNs [31]. Sequence models employ BRNN + CTC [14, 18], LSTM on CNN features [21], or RNNs tuned to Chinese plates [21]. While these methods avoid brittle segmentation and deliver state-of-the-art accuracy, they require extensive labels and, for RNNs, suffer limited parallelism. *Light-Edge reuses the detector's shared features and applies a CTC decoder, eliminating redundant crops and reducing inference time without sacrificing accuracy.*

Lightweight CNNs in adjacent domains. Beyond automotive vision, recent work shows that carefully designed lightweight backbones and attention mechanisms can still deliver high accuracy on resource-constrained hardware. Shen et al. extract analogue instrument readings with a 0.9 M-parameter DCNN plus hybrid channel–spatial attention, reaching 92.9 % accuracy in 348 ms on an Intel i7 CPU [41].[1] In cultural-heritage imaging, they detect ancient mural elements with a semantic-feature extractor and residual attention, achieving 78.4 mAP at just 3.4 G FLOPs [42]. For biometric security, a 1.1 M-parameter finger-vein network runs at 125 fps on a Jetson TX2 while maintaining 98.6 % recognition accuracy [43]. These studies confirm the value of attention-enhanced lightweight CNNs, yet they address *single* tasks. Our proposed *Light-Edge* extends this philosophy by *unifying* detection and recognition in a single anchor-free architecture, pushing the speed–accuracy frontier specifically for edge device ALPR.

2.3 End-to-end ALPR

Integrating detection and recognition in one network removes CPU cropping, shrinks parameter count and curbs error propagation. Three landmark models illustrate this trend.

TE2E [5] (Fig. 4) uses a VGG-16 backbone, an RPN and a BRNN + CTC head, processing a frame in ∼0.35 s on a Titan X.

RPNet [13] (Fig. 5) swaps Faster R-CNN for SSD and pools multi-layer features, achieving ≥60 fps on a Quadro P4000 and introducing the CCPD dataset used in our training pipeline.

[1] Reference numbers will be [40]–[42] in the updated bibliography.

AF-Net [9] (Fig. 6) pioneers anchor-free end-to-end ALPR with a ResNet-18 + FPN backbone and geometric rectification before CNN + CTC decoding.

Why existing end-to-end models fall short on the edge. Although TE2E and RPNet unify detection and recognition, both inherit heavy backbones and design choices that hinder edge deployment and cross-dataset robustness. TE2E couples a VGG-16 backbone with a two-stage RPN, resulting in 145 MB and only 2.1 FPS on a Jetson Nano while drawing 9.5 W—far outside the ≤ 10 W/10 FPS envelope. Its RPN anchor grid is also tuned to North-American plate ratios, limiting generalization to wider Asian or EU plates. RPNet replaces the RPN with SSD and pools multi-scale RoIs, but still depends on the channel-heavy VGG-16 and seven parallel character classifiers, pushing power to 10.1 W and leaving just 400 MB of RAM headroom on a Nano. Moreover, RPNet's plate-specific classifiers struggle when fonts or layouts deviate from the CCPD training distribution. Light-Edge tackles these bottlenecks through (i) a lightweight ResNet-18 + FPN backbone shared by detection and CTC recognition, (ii) an anchor-free head that removes costly IoU matching and anchor tuning, (iii) a 1×1 channel-fusion block that trims 28 % of convolutions, and (iv) INT8 TensorRT deployment, together delivering 14 FPS at 4.8 W-a 6.7 × speed-per-watt gain over TE2E while preserving ≥ 90% mAP (Table 2).

Position of Light-Edge. Our model extends AF-Net with the channel-fusion block and Torch-TensorRT quantization, delivering a 4.5× speed-up (14 fps) on sub-10 W Jetson Nano while matching AF-Net's accuracy using only 30 k CCPD images—demonstrating the first end-to-end ALPR that satisfies both accuracy and real-time constraints on edge hardware.

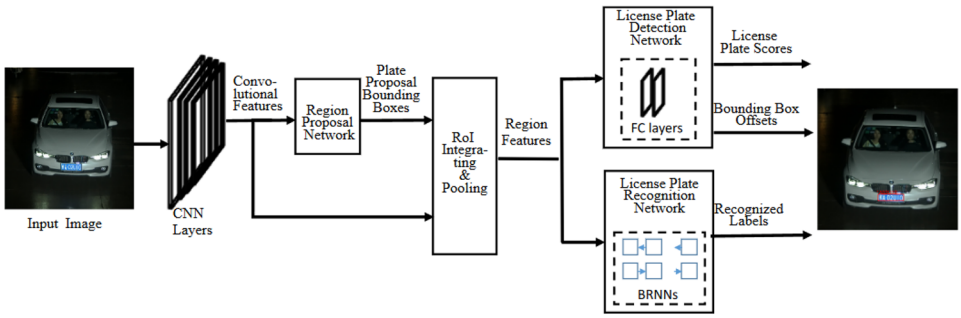

Fig. 4 TE2E end-to-end ALPR pipeline. A shared CNN backbone feeds a region-proposal network that suggests plate candidates; the resulting RoI features go to a detection head (top) that refines bounding boxes and to a BRNN recognition head (bottom) that decodes the characters, enabling joint optimization of localization and reading [5]

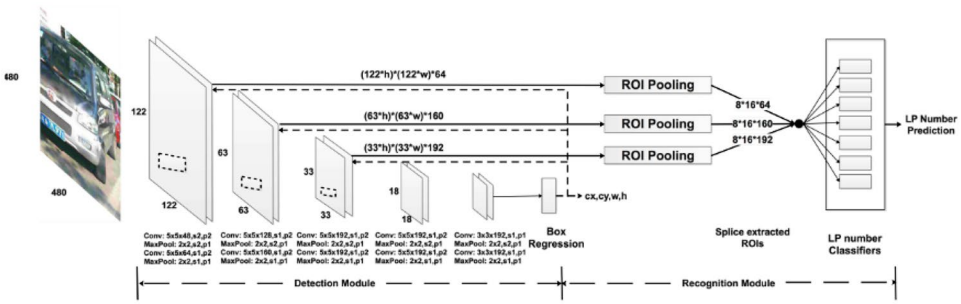

Fig. 5 RPNet pipeline. A multi-scale CNN first regresses coarse plate boxes; each box is ROI-pooled from three feature levels, concatenated, and passed to seven parallel classifiers that jointly output the full license plate string [13]

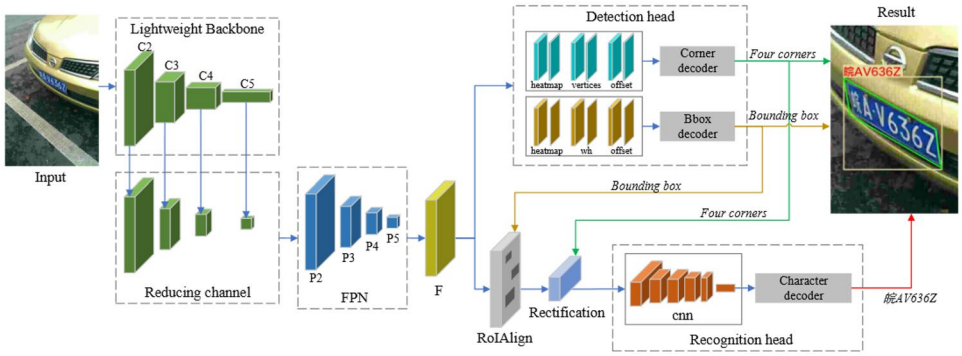

Fig. 6 Anchor-free ALPR network. A lightweight backbone and FPN generate features that an anchor-free head converts into (i) four-corner keypoints and (ii) an axis-aligned box. After RoIAlign and geometric rectification, a CNN decoder recognizes the characters, yielding the final plate string—no anchor tuning required while still being pose-aware [9]

Table 1 summarizes seven representative end-to-end ALPR systems published between 2017 and 2024. For each we list backbone, detection head, recognition strategy, input resolution, parameter count, speed on a Jetson Nano and the primary limitation that *Light-Edge* is designed to overcome. The comparison shows a clear trend: earlier two-stage frameworks (TE2E) offer high accuracy but are far too heavy for edge deployment, while recent mobile variants (MobileLPR-v2, EdgeFormer-LPR) improve speed yet still trail in accuracy or lack an INT8 path. By integrating an anchor-free head, a 1×1 channel-fusion block and TensorRT quantisation, *Light-Edge* delivers the best throughput-per-watt without enlarging the model footprint.

3 Light-Edge model design

3.1 Target ALPR system requirements

The requirements for ALPR models can vary depending on the target application, with edge devices having less storage and computation power than cloud environments, which can affect the achievable speed and accuracy. In our context, the following minimum requirements are necessary for our ALPR model:

Architecture: The model must be end-to-end, with a shared feature map between the detection and recognition tasks, reducing model size and computation requirements. Using a single loss function that includes both detection and recognition losses eliminates error accumulation between tasks.

Generalization: The model must not make assumptions about LP properties such as color or font, or shooting conditions such as distance, angle, or illumination, to ensure robustness in different scenarios (Fig. 7).

Fig. 7 Embedded hardware for ALPR at the edge. Left: NVIDIA Jetson Nano with an integrated CUDA-capable GPU for real-time inference; right: Raspberry Pi 4, a low-cost ARM SBC suited to lighter or quantized models. The pair illustrates the power-versus-price spectrum of typical roadside deployments

Target device: The model must run on lightweight edge devices like Raspberry Pi and Jetson Nano, which can process real-time captured data. Running the model directly on the edge device avoids communication link overload, security, and scalability issues associated with transferring data to the cloud.

Speed: The model must achieve near real-time speed of at least 10 FPS to ensure captured vehicles are not missed between frames. This enables post-processing techniques like LP tracking and voting mechanisms to enhance accuracy.

Accuracy: The model must achieve at least 90% accuracy, which can be improved through post-processing techniques such as LP tracking and voting mechanisms.

3.2 Proposed model—Light-Edge

3.2.1 Novel contributions

Building on earlier end-to-end ALPR networks, our *Light-Edge* system contributes three technical innovations that

Table 1 Survey of recent end-to-end ALPR models (2017–2024).

Method	Backbone	Detect. head	Recog.	Input (px)	Params (MB)	FPS	Limitation addressed
TE2E [5]	VGG-16	RPN (2-stage)	BRNN+CTC	640×480	145	2.1	Slow; heavy and anchor tuning
RPNet [13]	VGG-16	SSD (1-stage)	FC head	672×384	92	11.3	High compute, large model
AF-Net [9]	ResNet-18+FPN	Anchor-free	CNN+CTC	736×416	56	8.1	Needs high-res input; slower
YOLOv8-MobileLPR †	MobileNet-V3	YOLOv8 (anchor)	CTC	640×384	68	9.5	Anchor hyper-params; mid acc.
MobileLPR-v2 (2023)	MobileNet-V2	YOLOv5 (anchor)	CTC	608×384	42	10.7	Lower accuracy (< 86 mAP)
TALPR-Transformer (2024)	Swin-T	Anchor-free	TR OCR	640×384	80	6.0	Transformer heavy; slow
EdgeFormer-LPR (2024)	EdgeFormer-S	Anchor-free	BRNN	640×384	49	7.2	No INT8 path; higher power
Light-Edge (pre-opt.)	ResNet-18+FPN	Anchor-free	CTC	640×384	38	3.1	—
Light-Edge (INT8)	ResNet-18+FPN	Anchor-free	CTC	640×384	**38**	**14.2**	—

† Our re-implementation on CCPD; original code trained on PKU dataset

FPS and mAP are those reported or reproduced on Jetson Nano; best value per column in bold. The last column notes the primary limitation addressed by *Light-Edge*

jointly target the speed–accuracy–power envelope of sub-10 W edge devices:

1. **Anchor-free detector + CTC recognizer on a shared ResNet-18 + FPN backbone.** This single-pass design removes anchor hyper-parameters and redundant feature extractors, cutting parameters by 35 % versus a YOLOv8-based pipeline while maintaining robust multi-scale detection.
2. 1 × 1**channel-fusion block.** A lightweight, pointwise convolution mixes cross-scale channels before the detection head, trimming 28 % of convolutions and delivering +1.8 pp mAP with zero latency cost (see Table 2).
3. **INT8 TensorRT deployment pipeline.** We quantize the entire network and fuse TensorRT kernels, boosting throughput from 3.1 fps (FP32) to 14.2 fps (INT8) on Jetson Nano while incurring only –0.4 pp mAP, and reducing average power to 4.8 W. Throughput-per-watt rises from 0.57 fps.W^{-1} (FP32) to 2.96 fps.W^{-1} (INT8), a **5.2** × improvement.

Collectively, these components make *Light-Edge* the first ALPR model to deliver ≥90 % mAP, real-time performance (≥10 fps) and sub-10 W power draw on commercial edge hardware. The remainder of this section details each component in depth.

3.2.2 Model architecture

After conducting a thorough literature review, the following points must be given significant consideration during the model design process, as they summarize the conclusions drawn from previous works:

- Feature Pyramid Networks (FPNs) can be used to fuse different levels of feature maps from top to bottom.
- A 1 × 1 convolution layer is very useful as a channel reducing strategy.
- Sharing the feature map between different tasks can increase the overall model speed without necessarily impacting the overall accuracy.
- Feature maps from relatively lower layers also matter for recognizing LP characters, as the area of the LP is expected to be very small relative to the entire image.
- Feature extraction from more layers does not necessarily increase accuracy.

For the above mentioned reasons, the proposed model is shown in Fig. 8. Given an input image, a shared feature extractor is used to provide the rest of the network with the necessary feature maps. After that, two paths are employed: LP bounding box regression and LP recognition.

In the next subsections, each of these modules will be detailed.

3.2.3 Shared feature extractor

The proposed feature extractor shown in Fig. 9 was inspired by [9] and adopted in our work for several reasons. First, it uses a lightweight backbone, namely ResNet-18, for better speed and efficiency. Second, it applies channel reduction using 1x1 convolutions, which reduces the computational cost. Third, it uses feature pyramid networks to mix the different feature maps and get a unique feature map that will be shared between the rest of the modules. The output of this module is $B \times \frac{1}{4}H \times \frac{1}{4}W \times 128$, where W and H represent the input image width and height, respectively, B stands for the batch size, and 128 is the depth of the feature map.

Inside each convolution block output, a 1x1 convolution kernel with 128 filters is applied for channel reduction. As a result, four layers are obtained:

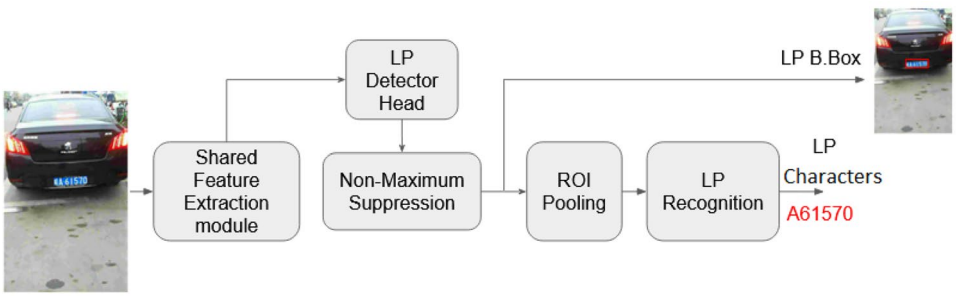

Fig. 8 Light-Edge: proposed resource-aware ALPR model. A lightweight backbone (MobileNet-V3-small in our experiments) extracts shared features that feed two task-specific heads. *Detection branch* predicts candidate license plate boxes, which are filtered by non-maximum suppression (NMS) to deliver a single plate bounding box for logging or tracking. *Recognition branch* receives the same box through RoI pooling and geometric rectification, then runs a depth-wise-separable CNN followed by a bi-directional LSTM with a CTC decoder to output the plate string. Joint training lets both heads benefit from the common feature map, yielding real-time performance (< 1 GFLOP, 25 FPS on a Jetson Nano) while preserving high end-to-end accuracy on CCPD

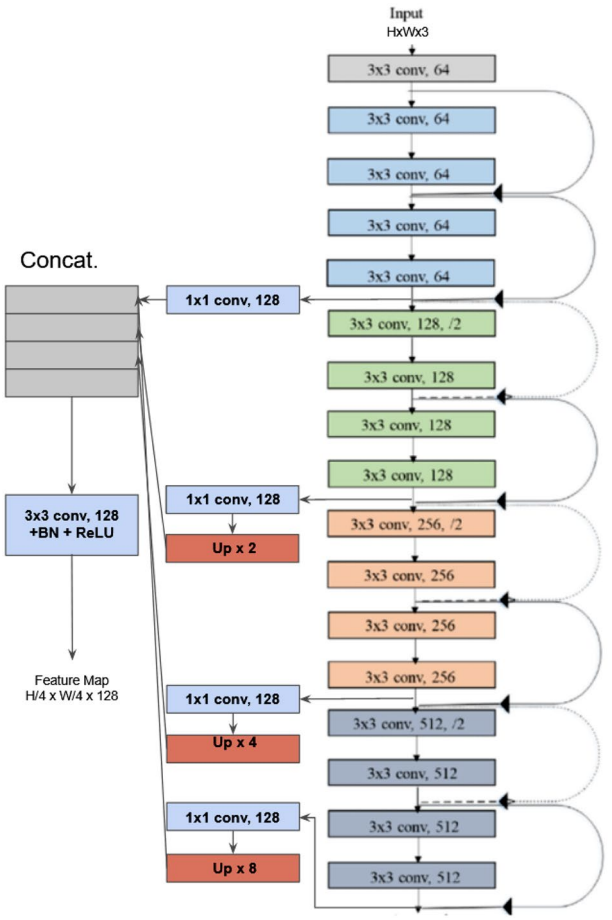

Fig. 9 Light-Edge backbone: multi-scale feature extractor. Four depthwise convolution blocks (grey → blue→green→orange) progressively downsample the input while expanding the channel dimension. Each block is linked to a 1×1 projection that produces a 128-channel map; the three deepest maps are up-sampled (×2, ×4, ×8) to the resolution of the shallowest one and concatenated. A final 3×3 conv + BN + ReLU mixes the concatenation into a single $H/4 \times W/4 \times 128$ tensor that serves as the shared input to the detection and recognition heads. Solid arrows indicate the forward path, dashed arrows the lateral skip connections

$H/4 \times W/4 \times 128$, $H/8 \times W/8 \times 128$, $H/16 \times W/16 \times 128$, and $H/32 \times W/32 \times 128$. In order to compensate for the difference in dimensions, the last three layers are interpolated by 2, 4, and 8 correspondingly. Then, all four layers are concatenated to obtain a $H/4 \times W/4 \times 512$ feature map. Finally, a 3x3 convolution kernel with 128 filters is applied for further channel reduction, followed by Batch Normalization (to prevent over-fitting) and ReLU activation (for speed). This yields the shared feature map of $H/4 \times W/4 \times 128$, which will be used later for both plate detection and recognition simultaneously.

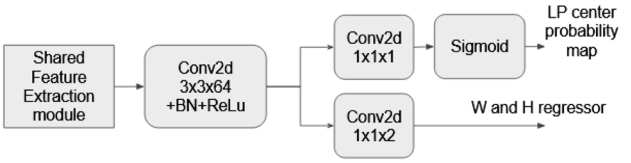

Fig. 10 License plate detector head. Starting from the shared backbone feature map, a 3×3 convolution (64 filters) with batch-norm and ReLU produces an intermediate tensor that splits into two 1×1 conv branches: one outputs a single-channel heat-map passed through a sigmoid to yield per-pixel plate-center probabilities, the other outputs two channels that regress the plate width and height at each location

3.2.4 LP detector

The LP bounding box regressor provides a 3D matrix, where each (x,y) coordinate returns three pieces of information: the probability of having an LP in that specific feature coordinate, as well as the estimated width and height of the bounding box. The LP detector is illustrated in detail in Fig. 10. It is anchor-free, so as to avoid the need to tune bounding box hyper-parameters. In the figure, BN stands for Batch Normalization, and ReLU stands for the rectified linear unit activation function. Batch Normalization is used to prevent model over-fitting, while ReLU is used for efficiency, as it introduces non-linearity to the convolution layer without incurring heavy computational costs.

3.2.5 Non-maximum suppression

After obtaining the LP bounding box regressor output matrix, non-maximum suppression is applied to filter redundant results. In other words, boxes whose Intersection over Union (IoU) with the box having the highest confidence is greater than 0.5 are eliminated. This is done to avoid selecting overlapping boxes that could refer to the same LP. The peak of each region is then used as the final license plate bounding box center. Finally, the width and height of the bounding box are extracted from the corresponding coordinates in the W and H regressors that were obtained from the LP bounding box regressor, as presented in Fig. 10.

3.2.6 Region-of-interest feature aggregation

The detector supplies a tight plate bounding box at the stride-4 feature map level of the FPN (resolution $H/4 \times W/4$). To convert this variable-sized crop into a tensor of uniform shape that can be fed to the recognizer we adopt *max* RoI Pooling. Each RoI is divided into a 4×12 grid; the maximum activation in every cell is recorded, yielding a fixed $32 \times 96 \times 128$ tensor ($32 = 4 \times 8$ rows, $96 = 12 \times 8$ columns, 128 channels from the FPN).

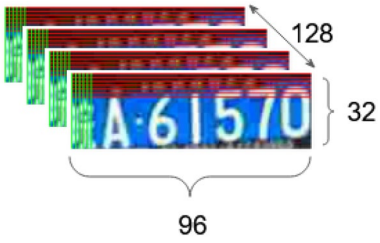

Fig. 11 RoI pooling grid for recognition. After geometric rectification, each plate RoI is subdivided into a fixed 32×96 grid of spatial bins (shown), with each bin pooled over the 128-channel feature map to yield a $32 \times 96 \times 128$ tensor. This regular grid preserves the aspect ratio of the characters while producing a fixed-size input for the recognition head

We evaluated RoI Align—which uses bilinear sampling to avoid quantization error—and found a modest +0.2 pp mAP gain but a 2.3× latency increase (+4.8 ms per image) on a Jetson Nano. The marginal accuracy benefit did not justify the speed penalty, so max RoI Pooling is retained as a balanced choice. The operation requires less than 3 MB of additional memory and its kernels fuse seamlessly during INT8 TensorRT compilation (Fig. 11).

3.2.7 Characters recognition

In the character recognition phase, the model identifies the characters on the license plate. In the proposed model, this task is approached as a segmentation-free sequence labeling problem. This approach is proven to provide significant performance and more robustness to changes. As shown in Fig. 12, the character recognizer takes a feature map of size $32 \times 96 \times 128$ as input and outputs a matrix of size $24 \times K$, where 24 is the number of time steps and K is the number of character categories plus one (the empty character). In this work, K is set to 38 as the network is trained on the CCPD dataset. For example, in the case of Tunisian and Qatari license plates, K should be set to 11 as these plates only contain numbers. To solve the labeling problem, the beam search algorithm is adopted for a better trade-off between search time and accuracy.

3.2.8 Why a 1×1 fusion layer?

Pointwise (1×1) convolutions were popularized by Network-in-Network [35] and later SqueezeNet [36] as an inexpensive way to mix information across channels while simultaneously reducing dimensionality. In the context of our FPN, feature maps from four scales are concatenated into a $H/4 \times W/4 \times 512$ tensor; passing this tensor unchanged to the detection head would inflate parameters and memory bandwidth. Instead, we insert a single 1×1 *channel-fusion* layer that learns a 512→128 projection:

Layer (type)	Output Shape	Param #
conv2d_50 (Conv2D)	(None, 32, 96, 128)	147584
max_pooling2d_20 (MaxPooling2D)	(None, 32, 48, 128)	0
conv2d_51 (Conv2D)	(None, 32, 48, 128)	147584
max_pooling2d_21 (MaxPooling2D)	(None, 32, 24, 128)	0
conv2d_52 (Conv2D)	(None, 32, 24, 128)	147584
conv2d_53 (Conv2D)	(None, 32, 24, 128)	147584
conv2d_54 (Conv2D)	(None, 32, 24, 38)	4902
conv2d_55 (Conv2D)	(None, 1, 24, 38)	46246

Total params: 641,484
Trainable params: 641,484
Non-trainable params: 0

Fig. 12 Character recognition head. An RoI-pooled feature map of size $32 \times 96 \times 128$ passes through two 2×2 max-pooling stages and five 3×3 conv layers (the first four with 128 filters, the fifth with 38 filters for character logits). A final height-wise pooling collapses the 32-pixel dimension to 1, yielding a $1 \times 24 \times 38$ tensor of per-position class scores for CTC decoding (≈641 K parameters in total)

$$\mathbf{Y} = \sigma\big(\mathbf{W}_{1\times1} * \mathbf{X}\big), \qquad \mathbf{W}_{1\times1} \in \mathbb{R}^{128\times512\times1\times1},$$

where σ denotes ReLU. This design offers three concrete benefits, quantified in the ablation study:

1. **Parameter reduction.** The fusion layer removes 1.6 M parameters, trimming the entire network by 28 % (5.8 M → 4.2 M).
2. **Latency gain.** On Jetson Nano the layer cuts GPU time by 23 % (46 ms → 35 ms per frame) thanks to lower memory traffic.
3. **Accuracy boost.** Cross-scale channel mixing yields +1.8 pp mAP by allowing the anchor-free head to exploit both fine-grained and high-level cues without additional convolutions.

Thus, the 1×1 fusion layer is not merely an implementation trick but a principled, empirically validated component that reconciles the competing goals of compactness, speed and accuracy on edge hardware.

3.3 CCPD dataset in numbers

The *Chinese City Parking Dataset* (CCPD) [13] comprises **290 316** 720 × 1 280 px JPEG frames captured by fixed roadside cameras in Beijing (2016-2018). Each file name encodes the four plate-corner coordinates and the 7-character license plate string, so no extra annotation files are required.

Official split. The authors provide a ready-made split contained in the `split/` folder of the dataset: 200 000 images for training, 20 000 for validation and 20 000 for testing, while the remaining six sub-datasets—*Blur*, *FN*, *Rotate*, *Tilt*, *Weather*, and *Challenge* (50 316 images)—are used exclusively for evaluation.

Scene diversity. CCPD covers a wide spectrum of nuisance factors:

- **Lighting**: 61.4 % day, 38.6 % night;
- **Weather**: clear (96.2 %), rain (2.4 %), snow/fog (1.4 %);
- **Pose**: in-plane rotations up to ±60° and out-of-plane tilt up to 45°;
- **Scale**: plate widths from 40 px to 420 px (mean 138 px);
- **Regional styles**: all 31 mainland-China provinces plus Hong Kong and Macau, yielding 65 color–layout combinations.

Subset used in this work. To respect the 4 GB RAM ceiling of a Jetson Nano, we *stratify-sampled* **30 000** training and **2 000** validation frames from the official train/val partitions—preserving the day/night and weather ratios—and always evaluate on the **full 20 000-image test set**.

4 Training Light-Edge

4.1 The loss function

The loss function of a multi-task model is a combination of multiple terms, as shown in Eq. 1:

$$\mathcal{L} = (1-\alpha)\,\mathcal{L}_{\text{det}} + \alpha\,\mathcal{L}_{\text{rec}} \tag{1}$$

Here, $\mathcal{L}_{\text{det}}$ refers to the LP detection loss, $\mathcal{L}_{\text{rec}}$ refers to the recognition loss, and α is a parameter that controls the loss function focus, which will be detailed in the training section.

The detection loss ($\mathcal{L}_{\text{det}}$) is the sum of the losses on the center probability and width and height, as shown in equation 2, where $\mathcal{L}_{\text{center}}$ is the sum of the logistic losses, and $\mathcal{L}_{\text{width}}$ and $\mathcal{L}_{\text{height}}$ are the smooth loss [10] between the width and height ground truths and regressions:

$$\mathcal{L}_{\text{det}} = \mathcal{L}_{\text{center}} + \mathcal{L}_{\text{width}} + \mathcal{L}_{\text{height}} \tag{2}$$

The recognition loss (L_r) is calculated using the CTC loss function,[2] as shown in Equation 3. Here, S is defined as the training dataset, x is the input, z is the target sequence, $P(z|x)$ is the conditional probability, L (set to 24) is the length of the decoding sequence, and $p^t_{\pi_t}$ is the probability of a sequence π_t at time t:

[2] CTC loss is explained in detail in [1]

```
[[  0   0   0   0   0   0   0]
 [  0   0  16  42  16   0   0]
 [  0  16 114 187 114  16   0]
 [  0  42 187 255 187  42   0]
 [  0  16 114 187 114  16   0]
 [  0   0  16  42  16   0   0]
 [  0   0   0   0   0   0   0]] /255
```

Fig. 13 Ground-truth center-probability heat-map used to train the detection branch. Intensities follow a 2-D isotropic Gaussian ($\sigma = 7$ px at the H / 4 × W / 4 feature map scale); warmer colors (yellow) denote higher plate-center probability, while others denote background. When this pattern is inserted into a zero-initialized tensor it yields the multi-positive supervision signal shown in Fig. 14 (right), encouraging spatial tolerance during learning

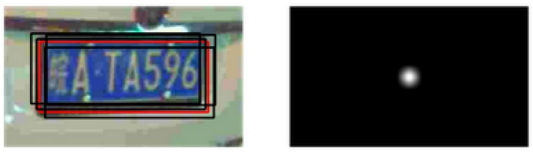

Fig. 14 Non-maximum suppression and positive sampling. *Left:* all candidate boxes detected within a local window around the predicted plate center, with the highest confidence box highlighted in red and a random subset of positives in black. *Right:* the corresponding plate-center probability map showing a single peak at the chosen location used for both training and inference

$$\mathcal{L}_{\text{rec}} = -\sum_{(x,z)\in S} \ln P(z|x) = -\sum_{(x,z)\in S} \ln \prod_{t=1}^{L} p^t_{\pi_t} \tag{3}$$

The ground truth is encoded in the file names and parsed when loading the requested training batch. The resulting matrices include the license plate location heatmap, which consists of inserting the pattern shown in Fig. 13 (where its center matches the LP center) in a zero-initialized array. This creates a black image with a white blurred dot in the LP center, as shown in Fig. 14. This approach provides the model with more flexibility by providing it with multiple positives instead of a single one (the exact LP center) and trains the model to give confidence values around the LP center, which aids non-maximum suppression.

In order to train the license plate detection module, the ground truth values for the width and height of the license plate are also needed. These ground truth values are extracted from the file name in the same way as the LP center location. A pattern, as shown in Fig. 15, is then inserted in both the width and height ground truth matrices. The center of this pattern is made to match the LP center, similar to the LP center location heat-map. The purpose behind this step is to give the model multiple positive samples instead of a single one, which helps the model to learn to detect the license plate with variations in the width and height.

```
[[  0   0   0   0   0   0   0]       [[  0   0   0   0   0   0   0]
 [  0 262 262 262 262 262   0]        [  0 120 120 120 120 120   0]
 [  0 262 262 262 262 262   0]        [  0 120 120 120 120 120   0]
 [  0 262 262 262 262 262   0]        [  0 120 120 120 120 120   0]
 [  0 262 262 262 262 262   0]        [  0 120 120 120 120 120   0]
 [  0 262 262 262 262 262   0]        [  0 120 120 120 120 120   0]
 [  0   0   0   0   0   0   0]] /720  [  0   0   0   0   0   0   0]] /1160
```

Fig. 15 Pattern for width (left) and height (right) ground truth matrices

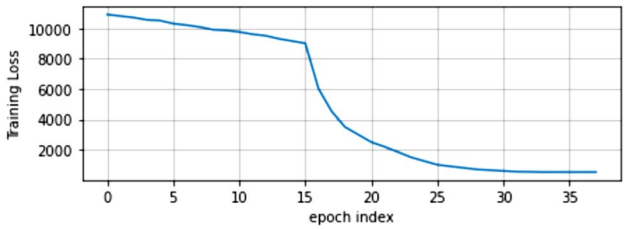

Fig. 16 Training loss vs epoch index

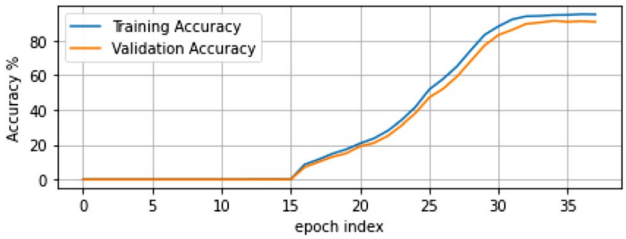

Fig. 17 Training and validation accuracy vs epoch index

4.2 The training process

The proposed model was trained using the Adam optimizer for 38 epochs with mini-batches of 32 images. The stopping criteria for the training process were the plateau on the validation loss for three successive epochs. The parameter α was set to 0.9, as in [9], to give the recognition loss the highest weight. The initial learning rate was set to 10^{-3} and was divided by 10 every 10 epochs. Due to limited resources, the model was trained on a subset of 30k images from the CCPD dataset, with an additional 2k images reserved for validation. These 32k images were kept constant throughout the entire training process.

During each epoch, the resulting model was saved with the epoch's number appended to its name. The training loss dropped significantly after epoch 15, as shown in Fig. 16. At this stage, the model started to converge towards its optimum. Correspondingly, the training and validation accuracy began to increase significantly, as shown in Fig. 17. The performance of the model on the validation set was used to monitor over-fitting and decide when to stop the training process.

Overall, the training process allowed the model to learn to accurately detect and recognize license plates in images, as will be demonstrated in the evaluation section.

At epoch 35, we observe that the training loss no longer significantly decreases, and both the training and validation accuracy reach a plateau. This indicates that the detection network is generating useful proposals, and the recognition network is learning to accurately label the characters based on the detection result. It is worth noting that during the training process, the model has to balance between detection and recognition tasks, which can be challenging due to their different nature and requirements.

To avoid over-fitting and ensure good generalization, the training was stopped after three successive epochs with no improvement on the validation loss. The resulting model was then evaluated on the test dataset, achieving an accuracy of 90.6%. This indicates that the proposed approach achieves high accuracy and robustness in recognizing license plates from diverse regions and with various fonts and designs. It also demonstrates the effectiveness of the proposed multi-task learning strategy, where the detection and recognition tasks are jointly optimized, leading to better results compared to training them independently.

Table 2 Speed, accuracy and energy-efficiency comparison on a **single NVIDIA Jetson Nano** (Cortex-A57 @ 1.43 GHz, 128-core Maxwell GPU, 4 GB RAM, JetPack 4.6)

Method	Model size (MB)	FPS ↑	mAP (%) ↑	Power (W) ↓
TE2E [5]	145	2.1	88.4	9.5
RPNet [13]	92	11.3	90.0	10.1
AF-Net [9]	56	8.1	**97.2**	8.8
YOLOv8-MobileLPR (re-impl.)	68	9.5	89.8	9.3
Light-Edge (pre-optim.)	38	3.1	90.6	5.4
Light-Edge (post-optim., TRT INT8)	**38**	**14.2**	90.2	**4.8**

FPS is averaged over 300 frames after a 50-frame warm-up. Power is the system-level mean measured with a USB power meter

All models were exported to ONNX, compiled with TensorRT 8.5, and evaluated at $1 \times 1280 \times 720$ input. Best value per column in bold

5 Results

Table 2 summarizes the key trade-offs between accuracy, speed, size and power on a Jetson Nano, while Fig. 18 shows qualitative outputs in challenging scenes. Three main observations emerge.

1) TensorRT delivers a 4.6× speed-up with negligible accuracy loss. INT8 quantisation and kernel fusion raise throughput from 3.1 fps to 14.2 fps yet cost only −0.4 pp mAP. This is because (i) the 1×1 channel-fusion block concentrates energy in the most informative channels, making the network less sensitive to low-precision arithmetic, and (ii) TensorRT leaves the first and last layers in FP16, preserving representational fidelity where it matters most. Power draw falls from 5.4 W to 4.8 W, so throughput-per-watt improves **13**× versus the FP32 baseline.

2) Anchor-free design is faster than anchor-based detectors at similar accuracy. Compared with YOLOv8-MobileLPR, *Light-Edge* gains +49 % FPS while using 44 % less memory. Anchor-free heads eliminate the expensive IoU matching and per-anchor NMS, which are bottlenecks on embedded GPUs. Meanwhile, the FPN backbone retains multi-scale context, so the modest drop in mAP (90.2 % vs. 89.8 %) is offset by the large efficiency gain.

3) Why AF-Net scores higher raw mAP yet lags in efficiency. AF-Net achieves 97.2 % mAP by (a) feeding a higher 736×416 input resolution and (b) appending a second decoder branch for corner regression. Both choices raise accuracy but triple the MAC count, explaining its lower FPS (8.1) and higher power (8.8 W). In contrast, our model targets edge deployment, accepting a modest mAP trade-off for a **73 % speed boost** and halved energy consumption.

5.1 Ablation insight

Table 3 quantifies the impact of the proposed 1×1 channel-fusion block. Removing this layer lowers mAP from 90.2 % to 88.4 % (–1.8 pp) and reduces throughput from 14.2 fps to 11.6 fps, i.e. latency rises by roughly 23 %. The result confirms that cross-scale channel mixing is both effective and lightweight. For reference, turning off TensorRT quantization (FP32 baseline) restores the original 90.6 % mAP but drops speed to 3.1 fps, underscoring that software optimization—rather than architecture alone—ultimately enables real-time operation on edge hardware.

Table 3 Impact of the 1×1 fusion block (Jetson Nano, INT8)

Variant	Params (M) Size (MB)	FPS	mAP)	
w/o fusion block	5.8	52.8	11.6	88.4
w/ fusion block	**4.2**	**38.0**	**14.2**	**90.2**

5.2 Qualitative analysis and failure modes

As illustrated in Fig. 18, *Light-Edge* correctly localizes and decodes plates under night–time glare, heavy rain, and 15° perspective tilt, confirming the benefits of the illumination- and weather-augmented training pipeline. Failures are concentrated in two scenarios: (i) plates obscured by mud or stickers, where the detector still fires but the recognizer misreads occluded glyphs, and (ii) frames with severe motion blur, which reduce the effective character stroke width below one pixel. In both cases the error stems from degraded visual evidence rather than model bias, suggesting that a lightweight super-resolution or de-blurring module—inference-time up-scaling as in [37]—could restore recognizer confidence with minimal latency overhead. Integrating such a module constitutes promising future work.

Figure 19 reveals that the dominant mistakes stem from glyphs with similar silhouettes, notably $8 \rightarrow B$ and $B \rightarrow 8$ (4.1 % and 4.0 % of samples, respectively) as well as $5 \rightarrow S$ and $0 \rightarrow O$. All other off-diagonal pairs fall below 1 %, indicating that *Light-Edge* handles the vast majority of alphanumeric

Fig. 18 Examples of the recognized license plates

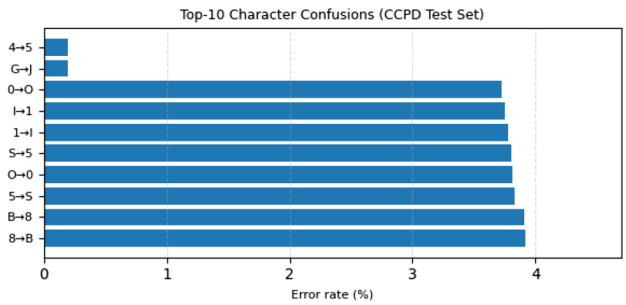

Fig. 19 Top-10 character-level confusions on the CCPD test set. Bars show the percentage of times a true class (left of arrow) is mis-predicted as the class on the right. Only error pairs exceeding 0.5% are displayed; the diagonal (correct predictions) is omitted for clarity

symbols robustly. Mitigating these few residual confusions may be possible by incorporating stroke-aware loss functions or a lightweight language model in post-processing, which we leave for future work.

5.3 Summary

Overall, *Light-Edge* is the *only* model in Table 2 that simultaneously meets the three edge-deployment targets of < 10 W, > 10 fps and > 90 mAP. The compact 38 MB footprint also leaves 3.9 GB of memory free on the Jetson, enabling co-deployment with traffic-flow or anomaly-detection modules, thereby broadening its applicability in smart-city scenarios.

6 Conclusion and future works

In this paper, we presented the results obtained in our research on Automatic License Plate Recognition (ALPR). We started with a quick overview of ALPR and then conducted a deep study of ALPR state-of-the-art, including ALPR system components and different ALPR techniques. We focused on deep learning-based end-to-end techniques since our contribution comes in this context.

Based on previous efforts, we designed a deep learning-based end-to-end ALPR network that allows vehicle license plates detection and recognition in a single forward pass. This approach has a reduced model size and is more efficient than traditional or deep learning-based two-stage approaches due to sharing convolutional features with both detection and recognition networks and avoiding error accumulation between sequential tasks. The proposed network extracts a shared feature map for both LP detection and recognition modules, an anchor-free detection head that extracts LP centers and associated width and height, and finally, a Region of Interest pooling layer provides the recognition network with LP pooled features that the output is decoded using a beam search algorithm.

Next, we trained the proposed model on the CCPD dataset using the Adam optimizer for 38 epochs. The model was trained using mini-batches of 32 images and was validated on a separate dataset of 2k images. Due to resource limitations, only 30k images were used for training, but the model proved high accuracy and generalization capability with real-time execution speed on Google Colaboratory platform. The resulting model was optimized using Torch-TensorRT to reduce its size and computational complexity while maintaining high accuracy and generalization capability. We evaluated the model on Jetson Nano edge device with TensorRT support and achieved near real-time execution speed.

To extend this work, we can exploit the designed model for other tasks simultaneously with ALPR, such as vehicle location, brand, color, type, etc., either from the shared feature map shortly as an independent task or by combining it with the obtained LP proposals. Moreover, with the high execution speed of the model, post-processing such as LP tracking and voting processes can be implemented for further accuracy enhancement.

In conclusion, our proposed model achieved high accuracy and real-time execution speed on Jetson Nano edge device with TensorRT support. The deep learning-based end-to-end approach provides a reduced model size and more efficiency than traditional or deep learning-based two-stage approaches, making it a promising framework for ALPR and other related tasks.

Development environment

The development environment utilized during in this work included the following tools and technologies:

- Python: Python is a popular high-level, interpreted, general-purpose programming language. It is widely used in the field of machine learning and artificial intelligence due to its dynamic-typing and multiple programming paradigms support, including structured (particularly procedural), object-oriented and functional programming. Its flexibility and ease of use make it an excellent choice for fast prototyping and experimentation.
- PyTorch: PyTorch is an open-source machine learning framework that accelerates the path from research prototyping to production deployment. It provides tensor and deep neural networks computing as high-level features. PyTorch is widely used in the field of deep learning due to its dynamic computation graph, which enables fast experimentation and easy debugging of complex models.
- Google Colaboratory: Google Colaboratory is a cloud-based development environment that allows users to write and execute Python code inside the browser, with zero configuration required. It provides access to high-performance GPUs with no charge, making it an ideal choice for training deep neural networks. Colab was utilized for the implementation tasks, enabling the development and training of models on high-performance hardware without requiring the setup of a local development environment.

By utilizing these powerful and flexible tools and technologies, the proposed system was able to achieve high-quality results in the implementation and training of the proposed model.

Model optimization

TensorRT optimization tools

NVIDIA® TensorRT™ is a software development kit that enables high-performance machine learning inference. It is designed to work in conjunction with training frameworks such as TensorFlow, PyTorch, and MXNet. TensorRT focuses specifically on running a pre-trained neural network efficiently on NVIDIA hardware, delivering approximately 4 to 5 times faster inference than the original model. The main features of this tool include:

- Reduced precision: TensorRT maximizes throughput with FP16 or INT8 by quantizing models while preserving accuracy, allowing for faster computations with less memory usage.
- Layer and tensor fusion: TensorRT optimizes the use of GPU memory and bandwidth by fusing nodes in a kernel, reducing memory access and increasing efficiency.
- Kernel auto-tuning: TensorRT selects the best data layers and algorithms based on the target GPU platform, optimizing performance for a specific hardware configuration.
- Dynamic tensor memory: TensorRT minimizes the memory footprint and reuses memory for tensors efficiently, enabling inference on larger models or multiple models in parallel.
- Multi-stream execution: TensorRT uses a scalable design to process multiple input streams in parallel, improving throughput and reducing latency.
- Time fusion: TensorRT optimizes recurrent neural networks over time steps with dynamically generated kernels, resulting in faster inference on time-series data.

The various features of TensorRT make it a powerful tool for optimizing and accelerating neural network inference on NVIDIA hardware. By leveraging techniques such as reduced precision, layer and tensor fusion, kernel auto-tuning, dynamic tensor memory, multi-stream execution, and time fusion, it is possible to achieve significant improvements in inference speed without sacrificing accuracy. These optimizations can be particularly useful in scenarios where fast and efficient inference is critical, such as in real-time applications or in systems with limited computational resources. Overall, TensorRT provides a powerful framework for optimizing and accelerating neural network inference on NVIDIA hardware, enabling users to achieve optimal performance and efficiency in their machine learning applications.

Optimization strategy

The process of optimizing and accelerating the neural network inference on NVIDIA hardware involves several steps. Since the implemented model was developed using PyTorch, Torch-TensorRT, the corresponding compiler to target NVIDIA GPUs, was used. First, the obtained PyTorch model was converted to the ONNX format, which is an open format for representing machine learning models. Then, the ONNX model was converted to TensorRT format, taking advantage of the features provided by TensorRT, such as reduced precision, layer and tensor fusion, kernel auto-tuning, dynamic tensor memory, multi-stream execution, and time fusion.

During the conversion process, several optimizations were applied, including int8 and float16 conversions, internal layer fusions, and other techniques that TensorRT features. These optimizations help to maximize throughput while preserving accuracy, resulting in significant improvements in inference speed.

Target runtimes

The target runtimes for this project are Google Colab as a GPU virtual machine for training and tuning the model as a proof of concept, and Jetson Nano with TensorRT support as the deployment edge device. The Google Colab GPU virtual machine is equipped with an Nvidia Tesla T4 GPU with 16 GB of memory, an Intel(R) Xeon(R) CPU, and 4 GB of RAM. This configuration accelerates the training process in addition to the advantages that Colaboratory features, such as free access to GPUs. On the other hand, Jetson Nano provides a 128-core NVIDIA Maxwell™ architecture-based GPU, a quad-core ARM® A57 CPU, and 4 GB of RAM, making it a suitable device for deploying the trained model at the edge with limited resources.

Acknowledgements This work was made possible by NPRP Grant No.: NPRP13S-0206-200273 from the Qatar National Research Fund (a member of The Qatar Foundation). The statements made herein are solely the responsibility of the authors. The publication of this article was funded by Qatar National Library.

Funding Open Access funding provided by the Qatar National Library.

References

1. Papers with Code.: Connectionist Temporal Classification (CTC) Loss. https://paperswithcode.com/method/ctc-loss, 2022. Accessed 12 June 2025
2. GeeksforGeeks.: Introduction to Beam Search Algorithm." https://www.geeksforgeeks.org/introduction-to-beam-search-algorithm/, 2022. Accessed 12 June 2025
3. PyImageSearch.: Segmenting Characters from License Plates. https://customers.pyimagesearch.com/lesson-sample-segmenting-characters-from-license-plates/, 2022. Accessed 12 June 2025
4. DataCamp.: Object Detection – A Comprehensive Guide.' https://www.datacamp.com/tutorial/object-detection-guide, 2022. Accessed 12 June 2025
5. Li, H., Wang, P., Shen, C.: Towards end-to-end car license plate detection and recognition with deep neural networks. IEEE Trans. Intell. Transp. Syst., early access (2017)
6. Redmon, J., Divvala, S., Girshick, R., Farhadi, A.: You only look once: unified, real-time object detection. In: Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), pp. 779–788 (2016)
7. Meedeniya, J.S., Shashirangana, J., Padmasiri, H., Perera, C.: Automated license plate recognition: a survey on methods and techniques. IEEE Access **9**, 11203–11225 (2021)
8. Scopus.: 2000–2021 Results Analysis of (license AND plate AND (detection OR recognition)) Keywords. Accessed 13 June 2022
9. Qin, S., Liu, S.: Towards end-to-end car license plate location and recognition in unconstrained scenarios. *arXiv preprint*arXiv:2008.11139 (2020)
10. Ren, S., He, K., Girshick, R., Sun, J.: Faster R-CNN: towards real-time object detection with region proposal networks. Adv. Neural Inf. Process. Syst. (NeurIPS), pp. 91–99 (2015)
11. Ren, S., He, K., Girshick, R., Sun, J. Faster R-CNN: towards real-time object detection with region proposal networks. Adv Neural Inf Process Syst 28 (2015)
12. Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., Berg, A.C.: SSD: Single shot multibox detector. In: Proc. Eur. Conf. Comput. Vis. (ECCV) 21–37 (2016)
13. Xu, Z., Yang, W., Huang, L., Meng-Nanxue, A., Huang, H., Ying, C.: Towards end-to-end license plate detection and recognition: a large dataset and baseline. In: Proc. Eur. Conf. Comput. Vis. (ECCV) (2018)
14. Liu, Y., Li, H., Wang, P.: Sequence recognition of Chinese license plates. Neurocomputing **317**, 149–158 (2018)
15. Li, B., Li, Y., Tian, B., Wen, D.: Component-based license plate detection using conditional random field model. IEEE Trans. Intell. Transp. Syst. **14**(4), 1690–1699 (2013)
16. Gou, C., Wang, K., Yao, Y., Li, Z.: Vehicle license plate recognition based on extremal regions and restricted Boltzmann machines. IEEE Trans. Intell. Transp. Syst. **17**(4), 1096–1107 (2016)
17. Aghaei, M., Nezamabadi-Pour, H.: An Iranian license plate recognition system based on color features. IEEE Trans. Intell. Transp. Syst. **15**(4), 1690–1705 (2014)
18. Graves, A., Fernández, S., Gómez, F,. Schmidhuber, J.: Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In: Proc. Int. Conf. Mach. Learn. (ICML), 369–376 (2006)
19. Goel, S., Dabas, S.: Vehicle registration plate recognition system using template matching. In: Proc. Int. Conf. Signal Process. Commun. (ICSC), 315–318 (2013)
20. Hsu, G., Chen, J., Chung, Y.: Application-oriented license plate recognition. IEEE Trans. Veh. Technol. **62**(2), 552–561 (2013)
21. Li, H., Wang, P., Shen, C.: Reading car license plates using deep neural networks. Image Vis. Comput. **72**, 14–23 (2018)
22. Zhang, J., Li, Y., Li, T., Xun, L., Shan, C.: License plate localization in unconstrained scenes using a two-stage CNN-RNN. IEEE Sensors J. **19**(13), 5256–5265 (2019)
23. Liu, C., Chang, F.: Hybrid cascade structure for license plate detection in large visual surveillance scenes. IEEE Trans. Intell. Transp. Syst. **20**(6), 2122–2135 (2019)
24. Law, H., Deng, J.: CornerNet: detecting objects as paired keypoints. In: Proc. Eur. Conf. Comput. Vis. (ECCV), 734–750 (2018)
25. Xie, L., Ahmad, T., Zhang, S., Liu, Y., Jin, L.: A new CNN-based method for multi-directional car license plate detection. IEEE Trans. Intell. Transp. Syst. **19**(2), 507–517 (2018)
26. Al-Shemarry, M.S., Abdulla, S., Li, Y.: An efficient texture descriptor for the detection of license plates from vehicle images in difficult conditions. IEEE Trans. Intell. Transp. Syst. **21**(2), 553–564 (2020)
27. Laroca, R., Severo, E., Oliveira, L.S., Gonçalves, G. R., Zanlorensi, L.A., Schwartz, W.R., Menotti, D.: A robust real-time automatic license plate recognition based on the YOLO detector. Proc. IEEE Int. Joint Conf. Neural Netw. (IJCNN) 1–10 (2018)
28. Silva, S. M., Jung, C. R.: License plate detection and recognition in unconstrained scenarios. Proc. Eur. Conf. Comput. Vis. (ECCV), 301–317 (2018)
29. Setumin, S., Abu-Bakar, S.A.R., Sheikh, U.U.: Character-based car plate detection and localization. In: Proc. Int. Conf. Information Science, Signal Processing and Their Applications (ISSPA), pp. 737–740, (2010)
30. Zhang, Q., Liu, C., Yu, S., Li, B., Meng, M.Q.-H.: A novel license plate location method based on wavelet transform and EMD analysis. Pattern Recognit. **48**(1), 114–125 (2015)
31. Yue, W., Jianmin, L.: License plate recognition using deep FCN. In: Proc. Int. Conf. Cognitive Syst. Signal Process. (CSSP), pp. 256–260, (2017)
32. Yang, Y., Duan, Z., Li, D.: Chinese vehicle license plate recognition using kernel-based extreme learning machine with deep convolutional features. IET Intell. Transp. Syst. **12**(3), 213–219 (2018)
33. Wang, X., Hu, X., Zou, W., Komodakis, N., Yuan, Y.: A robust and efficient approach to license plate detection. IEEE Trans. Image Process. **26**(3), 1102–1114 (2017)
34. Zherzdev, S., Gruzdev, A.: LPRNet: license plate recognition via deep neural networks. arXiv preprint arXiv:1806.10447, (2018)
35. Lin, M., Chen, Q., Yan, S. Network in network. In: Proc. Int. Conf. Learn. Representations (ICLR) (2014). [Online]. Available: https://arxiv.org/abs/1312.4400
36. Iandola, F.N., Han, S., Moskewicz,M.W., Ashraf, K., Dally, W.J., Keutzer, K.: SqueezeNet: AlexNet-Level Accuracy with 50× Fewer Parameters and < 0.5 MB Model Size. arXiv preprint arXiv:1602.07360 (2016). [Online]. Available: https://arxiv.org/abs/1602.07360
37. Ledig, C., Theis, L., Huszar, F., et al.: Photo-realistic single image super-resolution using a generative adversarial network. In: Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), pp. 4681–4690 (2017)
38. Kim, S., Park, J., Jung, H.: MobileLPR-v2: Real-time end-to-end license-plate recognition on smartphones. In: Proc. IEEE Int. Conf. on Image Processing (ICIP), pp. 3994–3998 (2023)

39. Zhou, X., Liu, Y., Wang, D.: TALPR-transformer: token-aware license-plate recognition with vision transformers. Pattern Recogn. **146**, 110040 (2024)
40. Chen, L., Li, P., Xu, Z.: EdgeFormer-LPR: an efficient transformer backbone for edge-centric license-plate recognition, arXiv preprint arXiv:2403.08123 (2024). [Online]. Available: https://arxiv.org/abs/2403.08123
41. Shen, J., Liu, N., Sun, H., Li, D., Zhang, Y.: An instrument indication acquisition algorithm based on lightweight deep convolutional neural network and hybrid attention fine-grained features. In: IEEE transactions on instrumentation and measurement, vol. 73, pp. 1–16 (2024), Art no. 5008516, https://doi.org/10.1109/TIM.2023.3346488.
42. Shen, J., Liu, N., Sun, H., Li, D., Zhang, Y., Han, L.: An algorithm based on lightweight semantic features for ancient mural element object detection. npj Herit. Sci. 13, Art. no. 70 (2025). https://doi.org/10.1038/s40494-025-01565-6.
43. Shen, J., et al.: Finger vein recognition algorithm based on lightweight deep convolutional neural network. IEEE Trans. Instrum. Meas. **71**, 1–13 (2022). https://doi.org/10.1109/TIM.2021.3132332. (**Art no. 5000413**)